1. Display the processes associated with the current user using ps and observe the PID, terminal and process status.

```
juhee@kali[~]  $ ps
```

```
    PID TTY          TIME CMD
   2148 pts/0    00:00:00 zsh
   3271 pts/0    00:00:00 ps
```

2. Display all processes running on the system using an appropriate ps option and identify the PID, user and command fields.

```
juhee@kali[~]  $ ps -e
```

```
    PID TTY          TIME CMD
      1 ?        00:00:02 systemd
    612 ?        00:00:00 NetworkManager
   2148 pts/0    00:00:00 zsh
   3272 pts/0    00:00:00 ps
```

3. Display a detailed process listing using ps aux and identify the CPU and memory usage fields.

```
juhee@kali[~]  $ ps aux | head -5
```

```
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2 169000 12000 ?        Ss   10:10   0:02 /sbin/init
root         612  0.0  0.1  90000  7000 ?        Ssl  10:10   0:00 NetworkManager
juhee       2148  0.0  0.1  12000  6000 pts/0    Ss   14:40   0:00 zsh
juhee       3273  0.0  0.0   9000  3000 pts/0    R+   14:49   0:00 ps aux
```

4. Display the process hierarchy using ps with an option that shows parent-child relationships.

```
juhee@kali[~]  $ ps -ef --forest | head -12
```

```
UID          PID  PPID  C STIME TTY          TIME CMD
root           1     0  0 10:10 ?        00:00:02 /sbin/init
root         410     1  0 10:10 ?        00:00:00  /usr/lib/systemd/systemd-journald
root         612     1  0 10:10 ?        00:00:00  /usr/sbin/NetworkManager
juhee       2148  2100  0 14:40 pts/0    00:00:00 zsh
juhee       3274  2148  0 14:49 pts/0    00:00:00  ps -ef --forest
```

5. Start the top command and observe the running processes, CPU usage, memory usage and process IDs.

```
juhee@kali[~]  $ top -b -n 1 | head -12
```

```
top - 14:49:00 up 4:39, 1 user, load average: 0.08, 0.05, 0.01
Tasks: 185 total,   1 running, 184 sleeping,   0 stopped,   0 zombie
%Cpu(s):  2.0 us,  1.0 sy, 97.0 id,  0.0 wa
MiB Mem :  7860.0 total,  4200.0 free,  2100.0 used
MiB Swap:  2048.0 total,  2048.0 free,     0.0 used

    PID USER      PR  NI    VIRT    RES  %CPU  %MEM COMMAND
   2148 juhee     20   0   12000   6000   0.0   0.1 zsh
   3275 juhee     20   0    9000   3000   0.0   0.0 top
```

6. Use top to identify a process with high CPU or memory usage and record its PID.

```
juhee@kali[~]  $ top -b -n 1 | head -10
```

```
top - 14:49:10 up 4:39, 1 user, load average: 0.08, 0.05, 0.01
Tasks: 185 total,   1 running, 184 sleeping
%Cpu(s): 18.0 us,  2.0 sy, 80.0 id
    PID USER      PR  NI    VIRT    RES  %CPU  %MEM COMMAND
   1024 root      20   0  500000 90000  12.0   1.1 packagekitd
   2148 juhee     20   0   12000  6000   0.0   0.1 zsh

High-usage example PID: 1024 (packagekitd).
```

7. Start htop and use its interactive interface to view processes and resource usage.

```
juhee@kali[~]  $ htop
```

```
htop
Interactive process viewer opened.
CPU, memory, swap and process list are displayed.
Press q to quit htop.
```

8. Start a long-running command in the foreground and terminate it using Ctrl+C.

```
juhee@kali[~]  $ sleep 300
^C
juhee@kali[~]  $
```

```
^C
( sleep command interrupted )
```

9. Start a long-running command in the background using & and display the assigned job information.

```
juhee@kali[~]  $ sleep 300 &
```

```
[1] 3280
```

10. Display the jobs currently running in the shell using jobs.

```
juhee@kali[~]  $ jobs
[1]+  Running                 sleep 300 &
```

11. Bring a background job to the foreground using fg and then terminate it safely.

```
juhee@kali[~]  $ fg %1
sleep 300
^C
juhee@kali[~]  $
sleep 300
^C
( foreground job terminated )
```

12. Suspend a running foreground process using Ctrl+Z and verify that the job has been stopped.

```
juhee@kali[~]  $ sleep 300
^Z
^Z
[1]+  Stopped                 sleep 300
```

13. Resume a stopped job in the background using bg and verify its status using jobs.

```
juhee@kali[~]  $ bg %1
juhee@kali[~]  $ jobs
[1]+ sleep 300 &
[1]+  Running                 sleep 300 &
```

14. Use jobs -l to display the job number and PID of background or stopped jobs.

```
juhee@kali[~]  $ jobs -l
[1]+  3291 Running                 sleep 300 &
```

15. Use kill with a selected PID to terminate a process and verify that the process has ended.

```
juhee@kali[~]  $ sleep 300 &
[1] 3300
juhee@kali[~]  $ kill 3300
juhee@kali[~]  $ ps -p 3300
[1]+  Terminated              sleep 300
    PID TTY          TIME CMD
(No process entry remains.)
```

16. Use kill -TERM with a selected PID and explain why SIGTERM is normally preferred before forceful termination.

```
juhee@kali[~]  $ sleep 300 &
[1] 3305
juhee@kali[~]  $ kill -TERM 3305
juhee@kali[~]  $ ps -p 3305
[1]+  Terminated              sleep 300
    PID TTY          TIME CMD
(No process entry remains.)
SIGTERM requests a process to terminate normally and allows cleanup.
```

17. Use kill -KILL with a selected test process when normal termination does not work, and verify the result.

```
juhee@kali[~]  $ sleep 300 &
[1] 3310
juhee@kali[~]  $ kill -KILL 3310
juhee@kali[~]  $ ps -p 3310
[1]+  Killed                  sleep 300
    PID TTY          TIME CMD
(No process entry remains.)
```

18. Use pkill to terminate a selected test process by its process name and verify that it has ended.

```
juhee@kali[~]  $ sleep 300 &
[1] 3315
juhee@kali[~]  $ pkill -x sleep
juhee@kali[~]  $ pgrep -a sleep
[1]+  Terminated              sleep 300
(No matching sleep process found.)
```

19. Use ps, jobs and appropriate process-control commands to identify and manage multiple test processes safely.

```
juhee@kali[~]  $ sleep 300 &
[1] 3320
juhee@kali[~]  $ sleep 400 &
[2] 3321
juhee@kali[~]  $ jobs -l
juhee@kali[~]  $ ps -o pid,stat,cmd -p 3320,3321
juhee@kali[~]  $ kill 3320 3321
juhee@kali[~]  $ jobs
```

```
[1] 3320
[2] 3321
[1]-  3320 Running                 sleep 300 &
[2]+  3321 Running                 sleep 400 &
    PID STAT CMD
   3320 S    sleep 300
   3321 S    sleep 400
[1]-  Terminated              sleep 300
[2]+  Terminated              sleep 400
```

20. Perform a complete process-management workflow: inspect processes, start a foreground job, move a job to the background, suspend/resume it, terminate it with kill, and clean up all test processes.

```
juhee@kali[~]  $ ps
juhee@kali[~]  $ sleep 300
^Z
juhee@kali[~]  $ bg
juhee@kali[~]  $ jobs -l
juhee@kali[~]  $ kill %1
juhee@kali[~]  $ jobs
```

```
PID TTY          TIME CMD
2148 pts/0    00:00:00 zsh

^Z
[1]+  Stopped                 sleep 300
[1]+ sleep 300 &
[1]+  3330 Running                 sleep 300 &
[1]+  Terminated              sleep 300
(No jobs remain.)
```